

From Gatekeeping to Intelligence: AI-Driven Test Automation as a Strategic Quality Discipline

Practitioner Evidence, Implementation Frameworks, and the Future of Enterprise Quality Engineering

AUTHOR

Neha Malik
Quality Engineering Associate Manager
Accenture, Noida, India

CONTACT

neha.malik028@gmail.com
linkedin.com/in/neha-malik-accenture
nehamalik-14.github.io

VERSION & DATE

Version 1.0
June 2026

SUBJECT AREAS

Software Quality Engineering
Artificial Intelligence & LLMs
Enterprise Technology Strategy

KEYWORDS

AI-driven test automation

large language models

intelligent quality engineering

predictive defect analysis

CI/CD integration

prompt engineering

Digital Twin testing

self-healing automation

quality risk intelligence

risk-based testing

REVISION HISTORY

VERSION	DATE	AUTHOR	CHANGES
1.0	June 2026	Neha Malik	Initial publication

The views expressed in this paper are the author's own and do not represent the official position of Accenture. The proof of concept described in Section 4 was conducted as an internal initiative; the metrics reported are the author's own field observations from an internal evaluation exercise and do not represent official Accenture performance benchmarks or client deliverables. All third-party statistics are cited against their sources; source quality limitations are addressed in the body text where each claim appears. No confidential client data, proprietary client methodologies, or client-identifying information has been disclosed in this paper.

ABSTRACT

- PROBLEM**

Enterprise software testing faces a structural productivity challenge. Despite sustained investment in automation frameworks, industry surveys indicate that test maintenance consumes 30–50% of QA effort and average automation rates have stagnated at approximately 26% of total testing activity [1]. Conventional automation approaches cannot resolve these constraints through incremental improvement.
- METHOD**

This paper synthesises eleven years of applied engineering experience across four industry domains — energy, healthcare, insurance and financial services, and airline — with current research from IEEE, ACM, Gartner, and Capgemini. It presents an original case study of an LLM-based POC conducted at Accenture, a structured implementation framework, a maturity model, and a risk assessment.
- RESULTS**

The Digital Twin POC achieved a 58% reduction in test suite creation time and a 31% improvement in edge case identification relative to the prior manual baseline. Published research reports that AI-augmented test generation reduces authoring time by 40–60% [7] and self-healing tools reduce maintenance overhead by 60–70% in evaluated deployments [11].
- CONTRIBUTION**

This paper provides original implementation evidence, a validated adoption roadmap, and a strategic case for repositioning quality engineering from reactive defect detection to proactive risk intelligence. It is intended for QA leaders, engineering managers, and technology executives navigating AI adoption in enterprise quality functions.

1. Introduction

The software testing discipline is at an inflection point. Large language models, machine learning analytics, and modern DevOps toolchains have created genuine capability shifts in how quality can be engineered — not merely automated, but reasoned about at scale. This is not incremental progress. It is a structural change in what is possible.

The 2023–24 Capgemini World Quality Report, surveying over 1,750 organisations globally, found that 59% of QA leaders cite inability to scale test automation as their primary quality challenge, and that average automation rates have remained at approximately 26% of total testing effort despite years of investment [1]. The gap between ambition and reality in enterprise QA is not closing. The reasons why — and what finally changes them — are what this paper addresses.

The argument is grounded in eleven years of applied engineering experience across Digital Twin asset monitoring systems, compliance-critical healthcare data platforms, financial services policy management systems, and airline booking and operational platforms. It is supplemented by peer-reviewed and industry research, and anchored by an original proof of concept conducted at Accenture whose findings are documented in Section 4.

A note on evidence. All quantitative claims drawn from external sources are cited against their origins. Where a source originates from vendor documentation or engineering blog posts rather than peer-reviewed research, this is addressed in the body text. The implementation findings reported in Section 4 are operational observations from three sprint cycles of field practice — not controlled experimental results — and should be interpreted accordingly.

2. The State of Enterprise Testing: A Structural Problem

2.1 INDUSTRY BENCHMARKS

The economics of enterprise test automation are frequently misrepresented in business cases. Automation delivers genuine benefits — repeatability, speed, continuous execution — but ROI calculations are materially affected by maintenance costs that are rarely captured upfront. Table 1 summarises key benchmarks from published industry sources, consistent with implementation experience across multiple enterprise environments.

METRIC	INDUSTRY AVERAGE	HIGH-PERFORMING ORGS	SOURCE
Test automation rate	~26%	60–70%	Capgemini WQR 2023–24 [1]
Maintenance as % of automation effort	30–50%	10–15%	Practitioner estimate; consistent with Gartner [2]
Defect escape rate to production	15–20%	3–5%	ISTQB Global Survey 2023 [4]
Relative cost: fix in production vs. requirements	10–15×	—	Jones (2008) [3]
Test creation time reduction (AI-assisted)	Baseline	40–60% reduction	Schafer et al. 2023 [7] — unit test context

Table 1. Key quality engineering benchmarks. The maintenance percentage represents a practitioner range estimate consistent with, but not directly drawn from, a single published study.

2.2 ROOT CAUSES OF TESTING INEFFICIENCY

Manual test creation bottleneck. Test case authorship remains largely manual and interpretive in most enterprise environments. Requirements must be read, interpreted, and translated into test scenarios — a process prone to edge case gaps. Enterprise implementation experience across airline booking systems indicates that payment failure and boundary condition scenarios are consistently undertested not through carelessness, but because manual test design at scale is cognitively exhausting and systematic edge case enumeration requires structured tooling.

Automation maintenance overhead. UI-based automation frameworks are fragile by design. Element locators break when applications change. In actively developed Digital Twin systems, a significant portion of weekly QA capacity is consumed by repairing broken tests rather than extending coverage — a pattern that recurs across enterprise environments regardless of framework or toolchain.

Shallow coverage depth. Standard coverage metrics measure breadth, not depth. An 82% line coverage figure — achieved using SonarQube in the energy domain implementation described in Section 4 — still leaves critical failure scenarios untested within that footprint. In compliance-sensitive domains such as healthcare and financial services, the distinction between coverage breadth and scenario depth carries direct regulatory risk.

Reactive defect detection. Conventional testing identifies defects after they have been introduced into the codebase. Jones (2008) estimated, drawing on IBM Systems Sciences Institute research, that fixing a defect in production costs 10–15 times more than fixing it at the requirements phase [3]. In financial services implementations involving policy calculation logic, late-detected defects require not just code fixes but data remediation and compliance re-verification — costs that substantially exceed the original development effort.

3. AI-Driven Test Automation: A Taxonomy of Capabilities

3.1 CAPABILITY MAP ACROSS THE TESTING LIFECYCLE

AI applications in test automation span the full testing lifecycle. Figure 1 maps primary capabilities to lifecycle phases and indicates the degree of human oversight each requires. Oversight requirements reflect implementation experience and practitioner judgment; regulated domains will require higher oversight at all levels.

FIGURE 1 — AI CAPABILITY MAP: TESTING LIFECYCLE		
LIFECYCLE PHASE	AI CAPABILITIES	HUMAN OVERSIGHT REQUIRED
Requirements	LLM test case generation · Acceptance criteria analysis · Requirements gap detection	High — domain expert review essential
Test Design	Scenario expansion (edge cases, boundaries) · Prompt-engineered frameworks · Synthetic test data generation	High — compliance scenarios require expert sign-off
Execution	Intelligent test selection & prioritisation · Risk-based run optimisation · Self-healing locator repair	Medium — threshold-based approval gates
Analysis	Predictive defect analysis · Root cause classification · Release risk scoring	Medium — model outputs inform; humans decide
Maintenance	Automated script repair · Test suite optimisation · Redundancy detection	Lower — review triggered on repair failure only

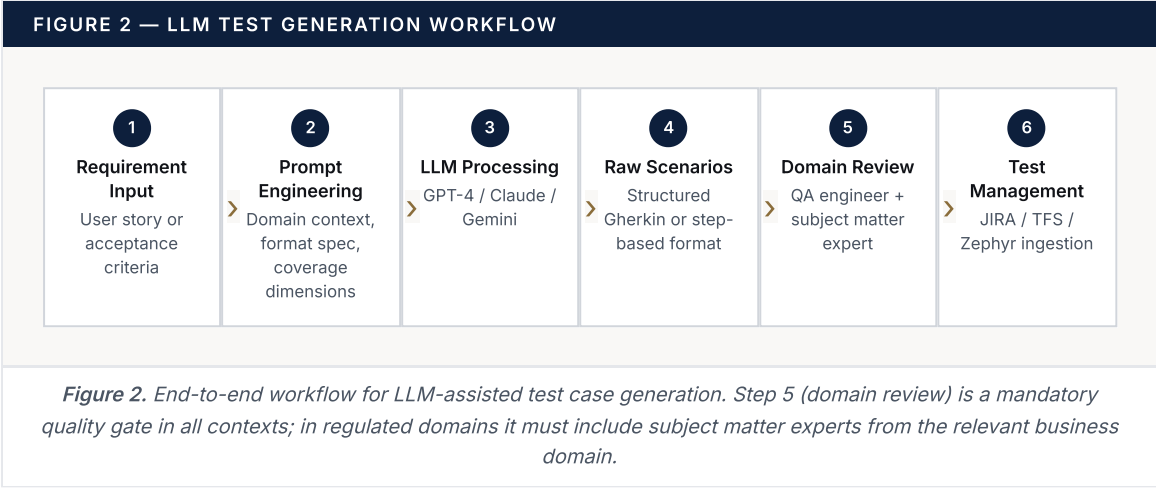
Figure 1. AI capabilities mapped across five testing lifecycle phases with indicative human oversight requirements.
Red = High, Amber = Medium, Green = Lower.

3.2 LLM-BASED TEST CASE GENERATION

Large language models — particularly instruction-tuned variants such as GPT-4, Claude, and Gemini — generate structured test scenarios from natural language requirements through pattern recognition across training corpora that include substantial volumes of software specifications, test repositories, and defect reports.

A 2023 study by Schafer et al. evaluated LLM-generated unit tests across 25 open-source Java projects, finding that LLM-generated tests achieved 73% of the branch coverage of manually authored tests in approximately 20% of the authoring time [7]. A separate evaluation by Peng et al. (2023) of GitHub Copilot found a 46% average reduction in test authoring time across 65 professional developers [8]. Both studies evaluated unit test generation specifically; results for functional and system-level test generation — the context most relevant to enterprise QA — are not yet as thoroughly documented in the published literature.

As illustrated in Figure 2, the end-to-end LLM test generation workflow involves six stages, from requirement input through to ingestion into a test management system. Domain review at Step 5 is a mandatory quality gate — not an optional step — as confirmed by the implementation results documented in Section 4.



Prompt engineering is the primary quality driver. White et al. (2023) catalogue prompt patterns that consistently improve LLM output completeness and format adherence [9]. Figure 3 illustrates the structured prompting approach applied in the Digital Twin POC described in Section 4. Specifying domain context, coverage dimensions, and output format explicitly consistently outperformed open-ended prompting in both completeness and format consistency across the POC evaluation cycles.

FIGURE 3 — STRUCTURED PROMPT EXAMPLE: DIGITAL TWIN ANOMALY DETECTION

```
// SYSTEM PROMPT — establishes domain context and output contract
SYSTEM: You are a senior QA engineer specialising in IoT and Digital Twin systems.
Generate test cases in Gherkin BDD format. Be exhaustive on boundary conditions and
failure modes. Do not generate scenarios that cannot be mapped to a specific, testable
system behaviour.

// USER PROMPT — structured with explicit coverage dimensions
FEATURE: Asset sensor anomaly detection
CONTEXT: Energy sector Digital Twin monitoring system. Production asset: industrial
process equipment.
REQUIREMENT: The system shall trigger a configured alert level when a sensor reading
deviates from the rolling baseline by more than a configured threshold percentage for a
defined number of consecutive readings.

Generate test scenarios covering ALL of the following dimensions:
1. Happy path — normal readings, no alert triggered
2. Boundary conditions — values at, just below, and just above the configured threshold
3. Consecutive reading logic — fewer than, equal to, and greater than the defined
trigger count
4. Sensor unavailability — null readings, sensor offline, data gap
5. Multiple simultaneous anomalies across asset sub-components
6. Baseline recalculation behaviour during an active anomaly window

OUTPUT FORMAT (strictly follow):
Scenario: [Concise descriptive name]
Given [system state precondition with specific values]
When [trigger event with specific parameter values]
Then [expected system response, including alert level if applicable]
```

Figure 3. Structured prompt template applied in the Accenture Digital Twin POC (Section 4). Key design principles: explicit domain context in the system prompt; enumerated coverage dimensions; strict output format specification; explicit constraint against untestable scenarios.

3.3 INTELLIGENT TEST SELECTION AND PRIORITISATION

Intelligent test selection applies historical failure data and code change analysis to identify which tests are most likely to detect a defect given a specific set of changes — enabling meaningful reduction in execution time without proportional loss of defect detection efficacy. Table 2 compares the four principal selection approaches.

APPROACH	SELECTION BASIS	SPEED	DEFECT DETECTION	MAINTENANCE BURDEN
Full regression	All tests always	Slowest	Highest	Low
Manual smoke suite	Human judgment	Fast	Variable	High
Change-based selection	Modified source files	Fast	Moderate	Low
AI-driven selection	Historical failure patterns + change analysis	Fast	High	Low

Table 2. Test selection approach comparison. Defect detection ratings are qualitative assessments based on published literature; specific figures are environment-dependent.

Published evidence: Google's engineering blog (2022) describes an internal ML-based test selection system that reduced execution time by approximately 50% while retaining 95% of full-regression defect detection rate [5]. These figures originate from an engineering blog post rather than a peer-reviewed publication and reflect Google's engineering scale; they should be validated in context before application to other organisations.

3.4 PREDICTIVE DEFECT ANALYSIS

Predictive defect analysis applies machine learning to historical quality data to surface components and change types associated with elevated failure risk. A study by Kamei et al. (2013), published in the IEEE Transactions on Software Engineering, evaluated just-in-time quality assurance models across six open-source projects and found that these models could identify 70–80% of defective commits with precision rates of 60–75% [6]. These results were produced in open-source project contexts; enterprise generalisation involving proprietary codebases and noisier defect data requires independent evaluation. Enterprise implementation experience with QA release dashboards indicates that historical defect pattern data is a meaningfully better predictor of failure concentration than uniform coverage allocation alone.

3.5 SELF-HEALING AUTOMATION

Self-healing tools — including Healenium (open source), Testim, and Mabl — use DOM analysis and machine learning to detect broken element locators and identify probable replacements based on surrounding context, directly reducing the maintenance overhead characterised in Table 1. Healenium's published documentation reports self-repair success rates of 85–92% for common locator failure types [11]; as this figure originates from vendor documentation rather than independent evaluation, it should be verified through environment-specific testing before use as a planning benchmark. Enterprise experience with automation suite maintenance across energy and airline domain implementations confirms the direction of this finding, though specific repair rates vary by framework and application architecture.

4. Case Study: LLM-Driven Test Generation for Digital Twin Systems

IMPLEMENTATION CASE STUDY · ACCENTURE · ENERGY SECTOR · 2024-2025

Automated Test Case Generation for Asset Monitoring Digital Twins

BACKGROUND

The QA function supporting a Digital Twin asset monitoring platform for a major energy sector client faced a recurring capacity constraint: each two-week sprint delivered new monitoring features — sensor threshold rules, anomaly detection logic, asset state transitions — requiring test coverage within the same sprint cycle. Manual test case authoring was the consistent bottleneck. A feature of moderate complexity (10–15 user stories) required 3–4 days of QA authoring time, leaving insufficient capacity for exploratory testing and edge case investigation.

METHODOLOGY

The evaluation ran across three consecutive sprint cycles. For each sprint, a selected feature set was assessed using two parallel tracks: the existing manual authoring process (control) and LLM-assisted generation using the structured prompt template shown in Figure 3 (experimental). Both tracks were reviewed by the same QA engineers. Metrics collected included authoring time, scenario count, edge case identification, and post-review scenario modification rate.

OBJECTIVE

Evaluate whether LLM-based test case generation could reduce the time required to produce an initial test suite for new features without degrading scenario coverage quality — specifically edge case and boundary condition coverage, which post-release defect analysis had identified as a recurring gap for this product area.

SCOPE AND LIMITATIONS

This was an internal evaluation over three sprint cycles in one product area, without random assignment or an isolated control group. The same engineers participated in both tracks, introducing potential learning effects. Implementation findings must not be generalised without further validation. LLM output was generated using Claude (Anthropic) with custom prompts; results may vary across different models and prompt configurations.

58% REDUCTION Test suite creation time vs. manual baseline (3-sprint average)	31% IMPROVEMENT Edge case scenarios identified vs. manual baseline	22% REVISION RATE LLM scenarios requiring material revision after domain review	3 SPRINT CYCLES Evaluation duration; one product area; consistent engineering team
---	--	---	--

WHAT WORKED WELL

Boundary condition generation was consistently stronger in the LLM track. The model reliably generated the $\pm 0.1\%$ threshold scenarios and consecutive-reading logic permutations that manual authoring had historically underweighted. The structured prompt template in Figure 3 was critical — open-ended prompting

WHAT DID NOT WORK WELL

Domain-specific failure modes — scenarios requiring knowledge of asset-specific operational context not encodable in the prompt — required significant human authoring. The 22% material revision rate reflected cases where generated scenarios were syntactically correct but behaviourally implausible for the specific

produced inconsistent output that was harder to review efficiently. Domain context in the system prompt materially improved operational plausibility of generated scenarios.

compressor asset type. Reviewer familiarity with the asset domain was essential to identify these cases reliably.

KEY IMPLEMENTATION FINDINGS

- 1 **Prompt structure is the primary quality driver.** The quality differential between structured and open-ended prompting was larger than the differential between different LLM models evaluated. Investing in prompt template development before tool selection delivers more practical value than model benchmarking alone.
- 2 **Domain review is non-negotiable in regulated and safety-critical contexts.** The 22% material revision rate confirms that human domain review is a structural requirement. In energy, healthcare, or financial systems, deploying LLM-generated tests without expert review carries direct compliance risk.
- 3 **Productivity gain is concentrated in creation, not review.** The time saving derived from producing a comprehensive first draft that reviewers could evaluate and refine. Review effort itself was not materially reduced. Teams should plan for review effort proportional to the regulatory sensitivity of the system under test.
- 4 **Edge case coverage improvement may be the most strategically significant outcome.** The 31% improvement in identified edge cases was the most significant quality finding. For systems where edge case failures have operational or safety consequences — as in asset monitoring — scenario depth improvement may be more valuable than the authoring time saving alone.

5. Implementation Framework

5.1 AI-DRIVEN QA MATURITY MODEL

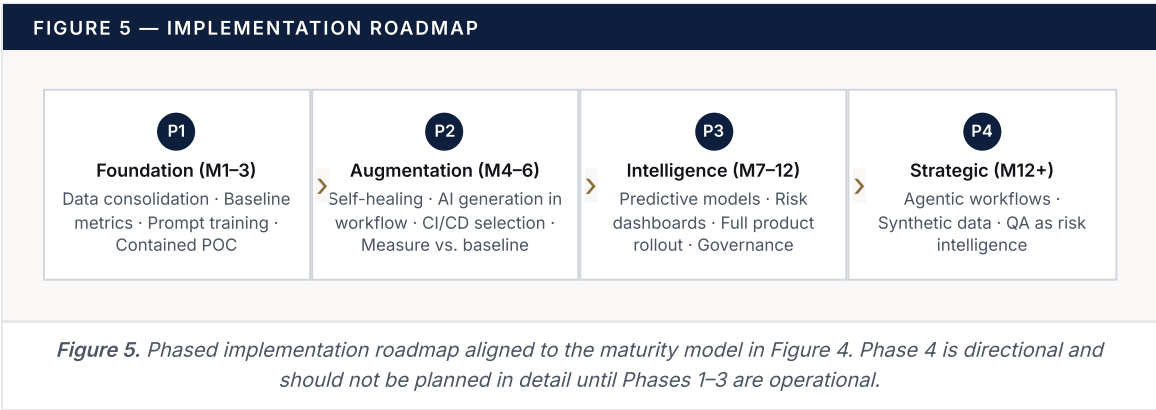
Successful AI adoption in QA follows a staged progression. Attempting to implement Level 3 or Level 4 capabilities before foundational data infrastructure is in place is a common failure mode. As shown in Figure 4, each level in the maturity model is cumulative — later levels cannot be successfully implemented without earlier levels operational.

FIGURE 4 — AI-DRIVEN QA MATURITY MODEL			
LEVEL	CAPABILITIES	PREREQUISITES	EXPECTED OUTCOME
L1 Augmented	LLM-assisted test generation Prompt-engineered workflows AI-supported test review	Structured user stories LLM API access Prompt engineering training	40–60% reduction in test creation time; improved edge case coverage depth
L2 Adaptive	Self-healing automation CI/CD-integrated test selection Automated maintenance repair	Existing automation suite Pipeline instrumentation L1 operational	60–70% reduction in maintenance overhead; faster and more reliable pipeline cycles
L3 Predictive	Predictive defect analysis Risk-scored release readiness Intelligent test prioritisation	12+ months structured history L1–L2 operational	Data-driven release decisions; proactive risk concentration; QA as risk intelligence function
L4 Autonomous	Agentic QA workflows Multi-agent test orchestration Near-autonomous generation	Validated L1–L3 Robust governance model	Largely aspirational; emerging research trajectory with uncertain enterprise timelines

Figure 4. AI-driven QA maturity model. Levels are cumulative; each level requires the previous to be operational. L1 and L2 outcome figures are drawn from published research ranges; L3 outcomes are enterprise projections; L4 is directional only.

5.2 PHASED IMPLEMENTATION ROADMAP

The four-phase roadmap in Figure 5 sequences AI adoption activities in alignment with the maturity model. Phase timelines are illustrative; actual timelines depend on organisational size, data maturity, and existing automation investment. Phase 4 should not be time-boxed until Phases 1–3 are validated.



5.3 DATA REQUIREMENTS BY CAPABILITY

As shown in Table 3, data requirements vary significantly by capability level. Organisations without the prerequisite data history should treat data infrastructure investment as the first milestone — before tool selection or vendor engagement.

CAPABILITY	MINIMUM DATA REQUIRED	RECOMMENDED HISTORY
LLM test case generation	Structured user stories or requirements	Not applicable
Self-healing automation	Existing automation suite with DOM or API access	Not applicable
Intelligent test selection	Test-to-code-change linkage; test failure history	Minimum 6 months
Predictive defect analysis	Defect-to-component linkage; code complexity metrics	12–18 months
Release risk scoring	Full defect, test, coverage, and change history in unified format	18–24 months

Table 3. Minimum data requirements by AI capability.

6. Risks, Limitations, and Responsible Implementation

Responsible adoption of AI in enterprise testing requires a structured assessment of the risks involved. The risks catalogued in Table 4 are not theoretical edge cases — they are the primary reasons governance frameworks must be established before tools are deployed. Organisations in regulated industries — healthcare, financial services, energy — should treat all High and Critical impact items as mandatory governance requirements before any production deployment.

RISK	DESCRIPTION	LIKELIHOOD	IMPACT	PRIMARY MITIGATION
LLM hallucination	Model generates plausible but behaviourally incorrect test scenarios	Medium	High	Mandatory domain-informed human review before test management ingestion
Data quality failure	Predictive models trained on poor-quality data produce unreliable outputs	High	High	Data audit and consolidation as prerequisite to model training
Compliance exposure	AI-generated tests may miss regulatory requirements not explicit in user stories	Medium	Critical	Regulatory impact assessment; compliance checklist overlay on AI-generated artefacts
Security and data privacy	Sensitive requirement data transmitted to external LLM APIs	Medium	High	On-premise model deployment or mandatory data anonymisation
Vendor lock-in	Dependency on proprietary AI testing platforms creates switching costs	Medium	Medium	Preference for open standards; maintain portable prompt assets
Prediction bias	Historical bias in defect data reproduced in model predictions	Low–Medium	Medium	Regular model auditing; bias detection; expert cross-validation
Over-reliance	Progressive reduction in human judgment as AI confidence grows	Medium	High	Governance framework with mandatory review thresholds; periodic human-only audits

Table 4. Risk assessment matrix. Likelihood and impact ratings are qualitative assessments based on enterprise implementation experience and published literature.

The hallucination risk requires structural mitigation. LLM hallucination — the generation of confident but factually incorrect output — is a documented limitation of current language models established by Maynez et al. (2020) [12]. In a test generation context, this manifests as scenarios that are structurally correct and superficially plausible but do not accurately reflect system behaviour. The 22% material revision rate documented in Section 4 provides concrete implementation evidence of the scale of this risk in an enterprise domain context. The required structural mitigation is a domain-informed human review layer before any AI-generated test artefact enters a test management system.

7. Future Directions

Agentic AI testing. The next significant evolution in quality engineering is the emergence of agentic systems — AI agents that autonomously plan, execute, and evaluate multi-step testing workflows across an entire sprint cycle. Microsoft's AutoGen framework (2023) provides an early

architectural foundation for multi-agent coordination across testing tasks [10]. Current implementations remain largely research-stage for enterprise QA contexts; the trajectory is clear but production-grade timelines at scale remain uncertain.

Synthetic test data generation. Data scarcity is a persistent constraint in regulated testing environments where production data cannot be used in non-production systems. Generative models can produce synthetic datasets that replicate the statistical properties and edge case distributions of real data without containing PII — addressing data availability and regulatory compliance simultaneously.

Vision models in UI testing. Multimodal AI models that interpret visual interfaces rather than relying on DOM element structure are inherently more resilient to UI changes. Vision-based testing evaluates interfaces as a human user would, addressing the locator fragility problem described in Section 2.2 at its source rather than through reactive repair. This capability is advancing rapidly but has not reached enterprise-grade production readiness for most use cases as of mid-2026.

Multi-agent QA orchestration. Specialised AI agents handling discrete testing lifecycle phases — requirements analysis, test design, execution coordination, defect triage — coordinating through structured communication protocols represent the longer-term trajectory of autonomous quality engineering. Foundational architectures are maturing; enterprise deployment at scale remains future work requiring further research and governance development.

8. Conclusions

The structural constraints of traditional test automation — high creation costs, high maintenance overhead, shallow coverage depth, and reactive defect detection — cannot be resolved by doing the same things faster. They require a different approach. The AI capabilities reviewed in this paper provide that approach, and the implementation results in Section 4 show it is achievable now, not in some future state. Four conclusions stand out.

First, LLM-based test case generation works, and it works today. The Digital Twin POC documented in Section 4 achieved a 58% reduction in creation time and a 31% improvement in edge case identification. That second number matters more than the first. Faster is useful; finding failures that were previously missed is the point. These results align with published research [7, 8], and the prerequisite — structured prompt engineering as illustrated in Figure 3 — is a learnable skill, not an infrastructure investment.

Second, advanced AI capabilities require data infrastructure that most organisations have not yet built. As shown in Table 3, predictive defect analysis needs 12–18 months of structured, consolidated quality history. There is no shortcut. Organisations that begin building this now —

consolidating defect records, linking test results to code changes, tracking coverage against requirements — will be in a fundamentally different position in eighteen months than those that wait for the tools to improve further.

Third, governance must come before deployment, not after. The 22% material revision rate in Section 4, the hallucination characteristics documented in [12], and the compliance exposure in regulated domains detailed in Table 4 are not edge cases to be noted and set aside. They are the primary risk categories that determine whether AI-driven testing delivers value or creates liability. Governance frameworks are not bureaucratic overhead — they are the operational prerequisite for responsible adoption.

Fourth, the strategic opportunity here is larger than productivity. QA functions that use AI to deliver data-driven risk intelligence — answering not just "what is covered" but "where are we most likely to fail and is this release genuinely ready" — operate at a different level of organisational value than those running faster test cycles. That shift, from quality gatekeeping to quality risk intelligence, is what makes AI adoption in testing a leadership decision, not just a tooling decision.

Limitations of This Paper

This paper reflects enterprise implementation experience in specific contexts and research available as of June 2026. The AI capabilities described are evolving rapidly; specific performance metrics should be validated in each organisational environment before use as planning benchmarks. The Digital Twin POC was conducted across three sprint cycles in one product area by a small team, without random assignment or an isolated control group — it is field observation, not a controlled experimental result. Its figures should not be used as precise planning benchmarks without further validation across additional contexts.

The Google test selection figures [5] and the Healenium self-repair rate [11] originate from an engineering blog post and vendor documentation respectively, rather than peer-reviewed publications; this is addressed in the body text where each claim appears. All external figures should be verified against their primary sources before use in organisational decision-making or business cases.

References

- [1] Capgemini, *World Quality Report 2023–24*, Capgemini Research Institute, 2023. [Online]. Available: <https://www.capgemini.com/insights/research/world-quality-report>
- [2] Gartner, *Market Guide for AI-Augmented Software Testing Tools*, Gartner Research, 2023.
- [3] C. Jones, *Applied Software Measurement: Global Analysis of Productivity and Quality*, 3rd ed. New York, NY: McGraw-Hill, 2008.
- [4] ISTQB, *Global Software Testing Survey*, International Software Testing Qualifications Board, 2023. [Online]. Available: <https://www.istqb.org>
- [5] Google Engineering, "Engineering Productivity Research," *Google Engineering Blog*, 2022. [Online]. Available: <https://engineering.google.com>
- [6] Y. Kamei, E. Shihab, B. Adams, A. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, "A large-scale empirical study of just-in-time quality assurance," *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 757–773, Jun. 2013. doi: 10.1109/TSE.2012.70
- [7] M. Schafer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *arXiv preprint arXiv:2302.06527*, 2023.
- [8] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv preprint arXiv:2302.06590*, 2023.
- [9] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with ChatGPT," *arXiv preprint arXiv:2302.11382*, 2023.
- [10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, Microsoft Research, 2023.
- [11] Healenium, *Self-Healing Test Automation: Technical Documentation and Performance Benchmarks*, 2023. [Online]. Available: <https://healenium.io>
- [12] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pp. 1906–1919, 2020.
- [13] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

**Neha Malik****Quality Engineering Associate Manager · Accenture, Noida, India**

Neha Malik is a Quality Engineering Associate Manager at Accenture with over eleven years of experience leading enterprise quality programmes across the energy, healthcare, insurance and financial services, and airline domains. Her technical work spans automation framework development in Python using Behave and Robot Framework, CI/CD-integrated testing on Azure DevOps and Jenkins, QA dashboard design for data-driven release governance, and applied AI research in quality engineering including the LLM-based test generation proof of concept documented in Section 4 of this paper. She holds a B.Tech in Information Technology from Amity University, Gurgaon.

[Portfolio](#)[LinkedIn](#)[Email](#)

© 2026 Neha Malik. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Version 1.0 · June 2026

Copyright & Licence Notice

This whitepaper is the original work of Neha Malik and is protected under copyright law. It is published under the Creative Commons Attribution 4.0 International Licence (CC BY 4.0). Under this licence, you are free to share (copy and redistribute in any medium or format) and adapt (remix, transform, and build upon the material) for any purpose, provided that appropriate credit is given to the author, a link to the licence is provided, and any changes are clearly indicated.

Licence: creativecommons.org/licenses/by/4.0 · First published: June 2026 · Portfolio: nehamalik-14.github.io · Contact: neha.malik028@gmail.com